

Relatório Técnico: Sistema de Reconhecimento de Pneumonia através de Imagens de Raio-X

Participantes:

- Luis Eduardo Alencar Melo
- João Pedro Coelho Barbosa

Pós-Graduação em Inteligência Artificial

Módulo: Redes Neurais Artificiais

Professor: Jackson Prado

1. Resumo Executivo

Este projeto implementa um sistema de classificação binária para diagnóstico automático de pneumonia através de análise de imagens de raio-X do tórax utilizando redes neurais convolucionais (CNN). O sistema desenvolvido é capaz de classificar automaticamente radiografias como "NORMAL" ou "PNEUMONIA" com alta precisão, proporcionando uma ferramenta de apoio ao diagnóstico médico.

2. Objetivo

Desenvolver um modelo de deep learning baseado em Convolutional Neural Networks (CNN) capaz de:

- Classificar automaticamente imagens de raio-X do tórax
- Distinguir entre casos normais e casos de pneumonia
- Fornecer visualização das regiões de interesse através de Grad-CAM
- Alcançar alta precisão e recall para auxiliar profissionais de saúde

3. Dataset Utilizado

Fonte: Chest X-Ray Images (Pneumonia).

Características do Dataset:

- Total de imagens:** 5.863 imagens de raio-X
- Classes:** 2 (NORMAL e PNEUMONIA)
- Formato:** JPEG
- Resolução:** Variável (redimensionada para 224x224 pixels)

- **Distribuição:**

- Treino: 5.216 imagens
- Validação: 16 imagens
- Teste: 624 imagens

Distribuição por Classes:

- PNEUMONIA: 4.273 imagens (73%)
- NORMAL: 1.583 imagens (27%)

4. Estrutura de Arquivos do Projeto

```
projeto_pneumonia/  
├── pneumonia_recognizer-copy.ipynb  
├── best_model.keras  
├── chest-xray-pneumonia/  
│   ├── versions/  
│   │   └── 2/  
│   │       ├── chest_xray/  
│   │       │   ├── train/  
│   │       │   │   ├── NORMAL/  
│   │       │   │   └── PNEUMONIA/  
│   │       │   └── val/  
│   │       │       ├── NORMAL/  
│   │       │       └── PNEUMONIA/  
│   │       └── test/  
│   │           ├── NORMAL/  
│   │           └── PNEUMONIA/  
└── requirements.txt
```

Arquivos Principais:

- `pneumonia_recognizer-copy.ipynb`: Notebook principal com todo o código
- `best_model.keras`: Modelo treinado salvo no formato Keras
- `chest-xray-pneumonia/`: Diretório contendo o dataset organizado

5. Metodologia e Arquitetura

5.1. Pré-processamento dos Dados

Redimensionamento: Todas as imagens foram redimensionadas para 224x224 pixels para padronizar o input da rede.

Normalização: Os valores dos pixels foram normalizados dividindo por 255.0, convertendo de [0,255] para [0,1].

Conversão de Tipo: Dados convertidos para float16 para otimizar o uso de memória.

5.2. Tratamento do Desbalanceamento

O dataset apresenta desbalanceamento significativo (73% pneumonia vs 27% normal). Para mitigar este problema:

Data Augmentation aplicada apenas na classe NORMAL:

- Rotação: ± 20 graus
- Deslocamento horizontal e vertical: 10%
- Zoom: 10%
- Flip horizontal: Ativado
- Modo de preenchimento: 'nearest'

Resultado pós-augmentation:

- PNEUMONIA: 2.905 amostras
- NORMAL: 2.095 amostras (originais + aumentadas)

5.3. Arquitetura da Rede Neural (Mini-VGG)

```
python
```

```
def build_mini_vgg(input_shape=(224, 224, 3)):
    model = models.Sequential([
        # Bloco 1
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Bloco 2
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Bloco 3
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Camadas Fully Connected
        layers.Flatten(),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid')
    ])
    return model
```

Características da Arquitetura:

- **Total de parâmetros:** Aproximadamente 3.2M
- **Função de ativação:** ReLU nas camadas convolucionais, Sigmoid na saída
- **Dropout:** 0.25 após cada bloco convolucional, 0.5 na camada densa
- **Pooling:** MaxPooling 2x2 para redução dimensional
- **Padding:** 'same' para manter dimensões

5.4. Configuração do Treinamento

Otimizador: Adam com learning rate inicial de 1e-4

Função de Perda: Binary Crossentropy (adequada para classificação binária)

Métrica: Accuracy

Callbacks Implementados:

1. **EarlyStopping:** Parada antecipada se val_loss não melhorar por 5 épocas
2. **ModelCheckpoint:** Salva o melhor modelo baseado em val_loss
3. **ReduceLROnPlateau:** Reduz learning rate por fator 0.5 se val_loss estagnar por 3 épocas

Pipeline de Dados:

- Batch size: 32
- Shuffle: Ativado no conjunto de treino
- Cache: Ativado para otimização
- Prefetch: tf.data.AUTOTUNE para melhor performance

6. Explicação Detalhada dos Blocos de Código

Bloco 1: Importação de Bibliotecas

```
python

import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
```

Importa as bibliotecas essenciais: TensorFlow para deep learning, Keras para construção do modelo, e Matplotlib para visualizações.

Bloco 2: Download do Dataset

```
python

import kagglehub
path = kagglehub.dataset_download("paultimothymooney/chest-xray-pneumonia")
```

Utiliza a API do Kaggle para baixar automaticamente o dataset de raio-X.

Bloco 3: Exploração da Estrutura

```
python

for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

Explora a estrutura de diretórios do dataset baixado para entender a organização dos arquivos.

Bloco 4: Função de Carregamento

python

```
def get_training_data(data_dir):  
    # Carrega e preprocessa imagens  
    # Redimensiona para 224x224  
    # Trata erros de carregamento
```

Função personalizada que carrega imagens, redimensiona e cria arrays NumPy com tratamento robusto de erros.

Bloco 5: Divisão dos Dados

python

```
X_train, X_temp, y_train, y_temp = train_test_split(  
    X, y, test_size=0.3, stratify=y, random_state=42  
)
```

Divide o dataset em treino (70%), validação (15%) e teste (15%) mantendo a proporção das classes.

Bloco 6-8: Data Augmentation

Implementa aumento de dados especificamente para a classe minoritária (NORMAL) para balancear o dataset.

Bloco 9: Pipeline de Dados

python

```
train_ds = tf.data.Dataset.from_tensor_slices((X_train_aug, y_train_aug))  
    .shuffle(1000).cache().batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
```

Cria pipelines otimizados do TensorFlow para carregamento eficiente dos dados.

Bloco 10-12: Construção e Compilação do Modelo

Define a arquitetura Mini-VGG e compila com otimizador Adam, função de perda binary crossentropy e métrica de acurácia.

Bloco 13: Callbacks

Configura callbacks para early stopping, checkpoint do melhor modelo e redução de learning rate.

Bloco 14: Treinamento

```
python
```

```
history = model.fit(train_ds, validation_data=val_ds, epochs=10, callbacks=callbacks)
```

Executa o treinamento do modelo por até 10 épocas com monitoramento de validação.

Blocos 15-16: Avaliação

Gera previsões no conjunto de teste, calcula matriz de confusão e relatório de classificação.

Blocos 17-22: Grad-CAM

Implementa Gradient-weighted Class Activation Mapping para visualizar quais regiões da imagem o modelo foca para fazer suas previsões.

7. Resultados Obtidos

7.1. Métricas de Performance

Matriz de Confusão:

```
..... Predicted
True ..... PNEUMONIA  NORMAL
PNEUMONIA ..... 390 ..... 16
NORMAL ..... 30 ..... 188
```

Relatório de Classificação:

```
..... precision recall f1-score support
PNEUMONIA ..... 0.93 ..... 0.96 ..... 0.94 ..... 406
NORMAL ..... 0.92 ..... 0.86 ..... 0.89 ..... 218
accuracy ..... ..... 0.93 ..... 624
macro avg ..... 0.92 ..... 0.91 ..... 0.92 ..... 624
weighted avg ..... 0.93 ..... 0.93 ..... 0.93 ..... 624
```

7.2. Análise dos Resultados

Pontos Fortes:

- **Alta Acurácia Geral:** 93% de acurácia no conjunto de teste
- **Excelente Recall para Pneumonia:** 96% (crucial para não perder casos positivos)
- **Boa Precisão:** 93% para pneumonia, 92% para normal

- **Baixo número de Falsos Negativos:** Apenas 16 casos de pneumonia classificados como normal

Pontos de Atenção:

- **Falsos Positivos:** 30 casos normais classificados como pneumonia
- **Recall para Normal:** 86% (pode ser melhorado)

7.3. Interpretabilidade com Grad-CAM

O Grad-CAM implementado permite visualizar quais regiões pulmonares o modelo considera mais importantes para a classificação, proporcionando:

- Transparência nas decisões do modelo
- Validação médica das regiões identificadas
- Confiança adicional para profissionais de saúde

8. Tecnologias e Ferramentas Utilizadas

8.1. Bibliotecas Python

- **TensorFlow/Keras:** Framework principal para deep learning
- **OpenCV:** Processamento de imagens
- **NumPy:** Computação numérica
- **Matplotlib/Seaborn:** Visualizações
- **Scikit-learn:** Métricas e divisão de dados
- **KaggleHub:** Download do dataset

8.2. Técnicas de Machine Learning

- **Convolutional Neural Networks (CNN)**
- **Data Augmentation**
- **Transfer Learning concepts (arquitetura VGG simplificada)**
- **Regularização (Dropout)**
- **Callbacks para otimização de treinamento**
- **Grad-CAM para explicabilidade**

9. Considerações Técnicas

9.1. Otimizações Implementadas

- **Uso de tf.data:** Pipeline otimizado para carregamento de dados

- **Cache e Prefetch:** Redução de gargalos de I/O
- **Float16:** Redução do uso de memória
- **Callbacks inteligentes:** Evita overfitting e otimiza hiperparâmetros

9.2. Tratamento de Desbalanceamento

A abordagem de data augmentation apenas na classe minoritária mostrou-se eficaz, evitando:

- Overfitting na classe majoritária
- Perda de informação importante
- Necessidade de técnicas mais complexas como SMOTE

10. Aplicações Práticas

10.1. Contexto Médico

Este sistema pode ser integrado em:

- **Sistemas hospitalares:** Triagem automática de radiografias
- **Telemedicina:** Suporte a diagnóstico remoto
- **Países em desenvolvimento:** Onde há escassez de radiologistas
- **Emergências:** Diagnóstico rápido em situações críticas

10.2. Limitações e Considerações Éticas

- **Não substitui profissionais:** Ferramenta de apoio, não diagnóstico final
- **Validação clínica necessária:** Testes em ambiente hospitalar real
- **Bias do dataset:** Limitado a populações específicas
- **Regulamentação:** Necessidade de aprovação por órgãos competentes

11. Trabalhos Futuros

11.1. Melhorias Técnicas

- **Ensemble de modelos:** Combinar múltiplas arquiteturas
- **Transfer Learning:** Utilizar modelos pré-treinados (ResNet, DenseNet)
- **Aumento do dataset:** Incluir mais variabilidade de casos
- **Multi-classe:** Classificar tipos específicos de pneumonia

11.2. Implementação

- **API REST:** Para integração com sistemas existentes

- **Interface web:** Para uso por profissionais de saúde
- **Mobile app:** Diagnóstico em campo
- **Validação clínica:** Estudos prospectivos em hospitais

12. Conclusões

O projeto demonstrou a viabilidade de utilizar redes neurais convolucionais para auxiliar no diagnóstico de pneumonia através de radiografias. Com 93% de acurácia e 96% de recall para casos de pneumonia, o modelo apresenta performance adequada para aplicação como ferramenta de apoio ao diagnóstico médico.

A implementação de Grad-CAM adiciona transparência crucial para a confiança médica, enquanto as técnicas de balanceamento de dados e regularização garantem robustez do modelo.

O sistema desenvolvido representa um exemplo prático de como a inteligência artificial pode contribuir para a democratização do acesso à saúde, especialmente em regiões com limitações de recursos especializados.

Principais Contribuições:

1. **Modelo robusto:** Alta performance com arquitetura eficiente
2. **Tratamento de desbalanceamento:** Técnica específica para dados médicos
3. **Explicabilidade:** Implementação de Grad-CAM para transparência
4. **Pipeline completo:** Da coleta de dados à implementação final
5. **Documentação técnica:** Código bem documentado e reproduzível

Data de Conclusão: Setembro de 2025

Tecnologia Principal: TensorFlow/Keras + Python

Tipo de Problema: Classificação Binária de Imagens Médicas